IBOOKTRACK · HANDOVER DOCUMENTATION

# Migration & Turnover Guide

A unified Warehouse Management System and Executive Dashboard for Innodata's book-digitization pipeline.

Generated 2026-08-09 · Ivan, Divan, John & Francis

# iBookTrack — Migration & Turnover Guide

doc migration audience DevOps difficulty medium

Step-by-step playbook for moving iBookTrack to a new environment (new PostgreSQL database, new server, new domain) **without losing data**.

## Migration at a glance




## 0. Pre-Flight

Before you start, gather these from the current owner:

- [ ] Access to the **current** PostgreSQL database (or a recent backup).
- [ ] The current `.env` file (or values for every variable in `PROJECT_OVERVIEW.md` §5).
- [ ] Repo access (Git).
- [ ] Domain DNS access if you're keeping the same URL.
- [ ] Innodata Simflex API key + config code (currently hard-coded — see cleanup notes).
- [ ] iLMS credentials if applicable.

## 1. Provision the New Database (PostgreSQL)

### 1a. Provision PostgreSQL

1. Spin up a new Vanilla PostgreSQL database on any provider (AWS RDS, DigitalOcean, local).
2. Save your connection string ( `DATABASE_URL=postgresql://user:password@host:port/db` ) in a vault.

### 1b. Run migrations on the new DB

Run **every** file in `backend/migrations/` in numeric order (001 → 025).

### 1c. Copy data from the old DB

Use `pg_dump` and `pg_restore` :

```
# Dump from source
pg_dump --dbname=<OLD_DATABASE_URL> --data-only --format=custom --file=ibooktrack.dump

# Restore into target
pg_restore --dbname=<NEW_DATABASE_URL> --data-only ibooktrack.dump
```

---

# 2. Deploy the Backend

The backend is a plain Node.js Express app — pick any host that runs Node ≥ 20.

## 2a. Render / Railway / Fly.io

1. New service → connect the repo.
2. **Root directory:** `backend/`
3. **Build command:** `npm install`
4. **Start command:** `npm start`
5. **Environment variables** — copy from your new `.env`:

   ```
   DATABASE_URL=postgresql://user:password@host:port/db
   PORT=5000
   FRONTEND_URL=https://app.example.com         # the deployed frontend
   FRONTEND_URLS=https://app.example.com,https://staging.example.com
   ```

6. Deploy. Hit `https://api.example.com/api/health` → expect `{ status: 'ok', timestamp: '…' }`.

## 2b. Self-hosted VPS

```
ssh root@server
git clone <repo> /opt/ibooktrack
cd /opt/ibooktrack/backend
npm ci --production
cp /opt/ibooktrack/.env.example /opt/ibooktrack/.env   # then edit

# Install pm2 (or systemd)
npm i -g pm2
pm2 start server.js --name ibooktrack-api
pm2 save && pm2 startup
```

Front it with **nginx** + a Let's Encrypt cert. Sample server block:

```
server {
  server_name api.example.com;
  listen 443 ssl;
  ssl_certificate     /etc/letsencrypt/live/api.example.com/fullchain.pem;
  ssl_certificate_key /etc/letsencrypt/live/api.example.com/privkey.pem;

  location / {
    proxy_pass http://127.0.0.1:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $remote_addr;
  }
}
```

---

# 3. Deploy the Frontend

## 3a. Vercel (recommended)

1. Import the repo. **Root directory:** `frontend/`.
2. Environment variables:

   ```
   NEXT_PUBLIC_API_URL=https://api.example.com
   ```

3. Build command: auto (`next build`).
4. Add the production domain.

> ⚠ **WARNING**
>
> **Casing fix before Linux deploy:** rename `frontend/app/Dashboard/` → `frontend/app/dashboard/` and update all 25 references (search the repo for `'/Dashboard'`). Vercel runs Linux; case-sensitivity **will** break routes on first deploy.

## 3b. Self-hosted Node

```
cd /opt/ibooktrack/frontend
npm ci
npm run build
pm2 start "npm start" --name ibooktrack-web
```

Reverse-proxy as above (port 3000).

---

# 4. Pre-Flight Checks (Smoke Tests)

After both services are live:

| Check | Expect |
|---|---|
| `curl https://api.example.com/api/health` | `{"status":"ok",…}` |
| `https://app.example.com/landing-page` loads | Marketing page |
| Register new account | Lands on `/Dashboard` |
| Login → token saved in `localStorage` | DevTools confirms |
| Dashboard loads stats + transmission + flagged tables | All three populate |
| `/warehouse-inventory` → start session → scan ISBN | Box auto-created in DB |
| Flag scan → reopens with proper label (not raw key) | UI uses `FLAG_LABELS` |
| `/checkpoints` → advance a box | `pipeline_events` row inserted |
| Sidebar log out | Tokens cleared, redirect to /login |

# 5. Rollback Plan

If the new deployment misbehaves:

1. **DNS:** revert `app.example.com` and `api.example.com` to the old IPs.
2. **Frontend:** Vercel keeps every deployment — promote the previous one.
3. **Database:** The old database should be left **read-only** for at least 7 days post-cutover. To go back, point `DATABASE_URL` at it in both frontend and backend env vars and redeploy.

# 6. Contact / Handoff

For the complete list of authors and contributors, see Team & Credits.

Built with Windsurf AI pair-programming. The conversation history with each AI session is preserved if you need to re-read design decisions.

Welcome aboard, and good luck. 🚀